



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет имени
Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «ИУ Информатика и системы управления»

КАФЕДРА «ИУ7 Программное обеспечение ЭВМ и информационные технологии»

Лабораторная работа № 6 по дисциплине «Анализ алгоритмов»

Тема Методы решения задачи коммивояжёра

Студент Еланская Е.Д.

Группа ИУ7-53Б

Преподаватели Волкова Л.Л., Строганов Ю.В.

Москва, 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Аналитическая часть	4
1.1 Алгоритм полного перебора	4
1.2 Муравьиный алгоритм	4
1.3 Задание по варианту	5
2 Конструкторская часть	6
2.1 Описание алгоритмов	6
2.2 Расчёт трудоёмкости алгоритмов	8
3 Технологическая часть	9
3.1 Реализация алгоритмов	9
3.2 Тестирование	12
4 Исследовательская часть	13
4.1 Параметризация муравьиного алгоритма	13
4.2 Замеры реального времени	13
ЗАКЛЮЧЕНИЕ	15
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	16
ПРИЛОЖЕНИЕ	17

ВВЕДЕНИЕ

Цель работы – сравнить способы решения задачи коммивояжёра.

Задачи, необходимые для достижения поставленной цели:

- 1) описать метод полного перебора для решения задачи коммивояжёра, указать его преимущество и недостаток;
- 2) описать метод решения задачи коммивояжёра на основе муравьиного алгоритма, указать его преимущество и недостаток;
- 3) описать схемы алгоритмов для обоих методов и выполнить оценку трудоёмкости составленных алгоритмов по разработанным схемам;
- 4) реализовать оба алгоритма;
- 5) выполнить параметризацию последнего метода по трём его параметрам;
- 6) по итогам выполненного исследования дать рекомендации о значениях либо диапазонах значений параметров, настройках и/или режимах работы метода, основанного на муравьином алгоритме;
- 7) провести сравнительный анализ двух рассмотренных методов решения задачи коммивояжёра.

1 Аналитическая часть

Задача коммивояжёра заключается в поиске минимального пути, при котором все узлы графа будут посещены [2]. Гамильтонов цикл – это модификация задачи коммивояжёра, добавляющая ещё одно условие к изначальной формулировке: начало обхода совпадет к его концом.

1.1 Алгоритм полного перебора

Суть алгоритма полного перебора лежит в его названии: все возможные пути графа последовательно перебираются, в результате находится либо самый оптимальный путь, либо его отсутствие [3].

1.2 Муравьиный алгоритм

Идея муравьиного алгоритма (Ant Colony Optimization, ACO) заключается в моделировании поведения колонии муравьёв, которые коллективно находят оптимальные пути, используя феромоны как механизм координации и обмена информацией [4].

Каждые муравьиные сутки (итерация алгоритма) каждый из m муравьёв помещается в случайную вершину i_k и начинает движение. Следующая вершина маршрута j выбирается в зависимости от вероятности (формула 1.1):

$$p_{ij}^k = \frac{\tau_{ij}^\alpha \cdot \eta_{ij}^\beta}{\sum_{l \in N_i^k} \tau_{il}^\alpha \cdot \eta_{il}^\beta} \quad (1.1)$$

где

- τ_{ij} – концентрации феромона на дуге из вершины i в вершину j ;
- $\eta_{ij} = \frac{1}{d_{ij}}$ – эвристика (исследование);
- α, β – коэффициенты влияния феромона и жадности соответственно;
- N_i^k – множество непосещённых соседей муравья k из вершины i .

По истечении суток полученные маршруты L_k сравниваются с результатами предыдущих суток, а концентрации феромонов обновляются по правилу (формула 1.2):

$$\tau_{ij} = (1 - \rho)\tau_{ij} + \sum_{k=1}^m \Delta\tau_{ij}, \quad (1.2)$$

где ρ – коэффициент испарения, а

$$\Delta\tau_{ij} = \begin{cases} \frac{Q}{L_k}, & \text{если муравей } k \text{ прошёл по дуге } ij, \\ 0, & \text{иначе.} \end{cases} \quad (1.3)$$

Q – максимальное количество феромона, которое вычисляется относительно длин рёбер рас-

смаатриваемого графа.

По истечении предельного количества суток возвращается итоговый результат – оптимальный путь графа.

1.3 Задание по варианту

Ор-граф, без элитных муравьёв, вокруг света за 80 дней (на рёбрах затраты по времени; есть предел – 80 дней в человеческом измерении, не совпадающем с муравьиным). Гамильтонов цикл.

Вывод

В данном разделе мной был проведен анализ теоретических материалов, объясняющих принцип работы алгоритмов.

2 Конструкторская часть

2.1 Описание алгоритмов

Схемы рассматриваемых алгоритмов представлены на рисунках 2.1 и 2.2.

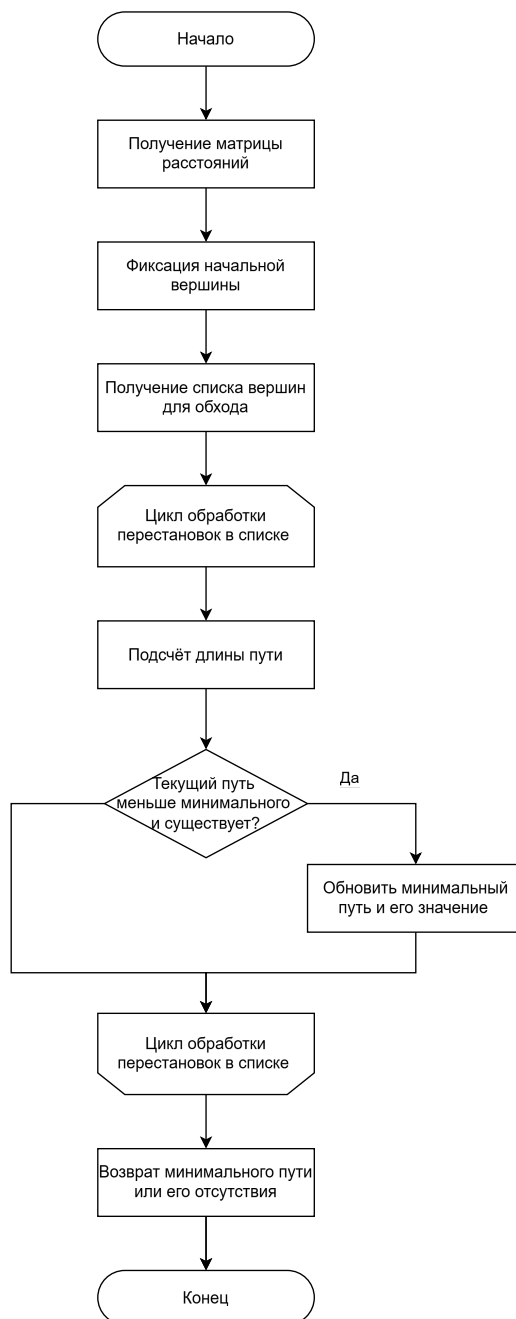


Рисунок 2.1 — Схема алгоритма полного перебора.

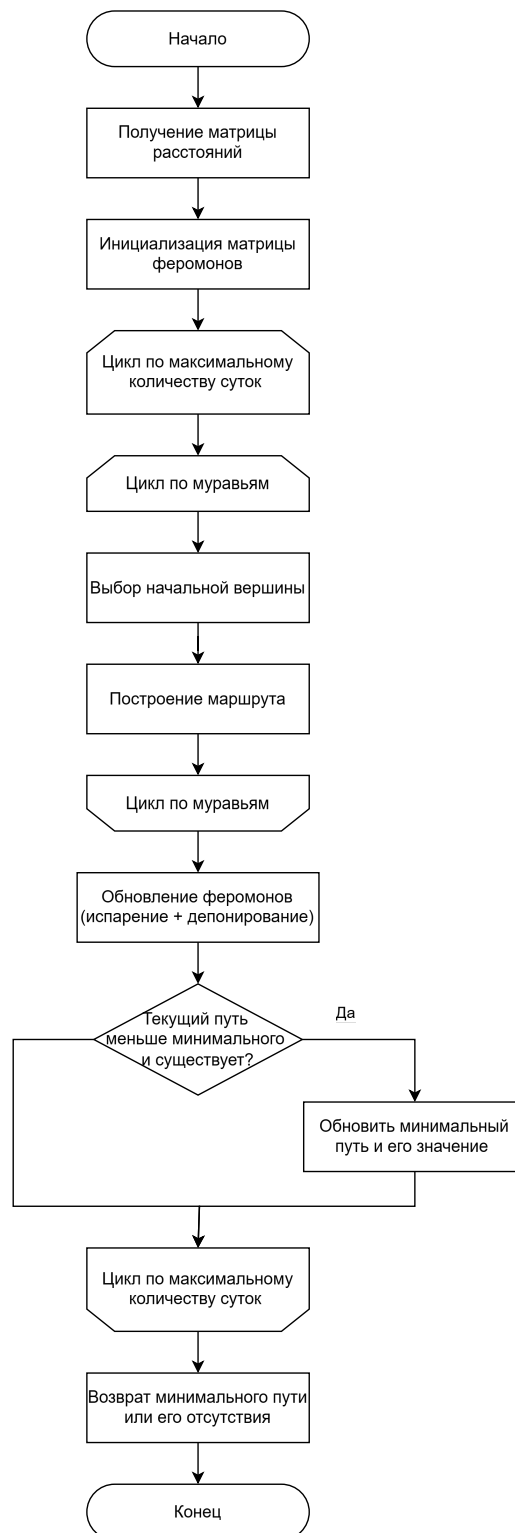


Рисунок 2.2 — Схема муравьиного алгоритма.

2.2 Расчёт трудоёмкости алгоритмов

С помощью введённой модели вычислений (табл. 2.1) была определена теоретическая трудоёмкость описанных ранее алгоритмов, результаты представлены в табл. 2.2.

Таблица 2.1 — Цены базовых операций.

Операции	Цена
$+, -, =, ++, --, \ll, \gg$	1
$*, /, \%, *=, /=$	2
$==, >=, <=, !=, >, <, \&\&, \parallel$	1
$[], \{ \}$	1

Таблица 2.2 — Трудоёмкость алгоритмов.

Алгоритм	Трудоёмкость ЛС	Трудоёмкость ХС
Полный перебор	$(n - 1)!(2n + 1) + n + 4$	$(n - 1)!(2n + 3) + 2n + 2$
Муравьиный	$(T_{max} - 1)n^3 + 3T_{max}n + 3T_{max} + 2$	$T_{max}n^3 + (2T_{max} + 2)n + 2T_{max} + 5$

Вывод

В данном разделе мной были построены схемы алгоритмов и вычислена их трудоёмкость с помощью введённой модели вычислений.

3 Технологическая часть

3.1 Реализация алгоритмов

Для реализации алгоритмов использовался язык программирования C++, так как он поддерживает статическую типизацию и имеет возможность отключения сборщика мусора. Отладка происходила в среде разработки «Visual Studio 2022».

```
1 bool my_next_permutation(std::vector<int>& nodes){
2     bool res = true;
3     int n = nodes.size(), i, j;
4     for (i = n - 2; i >= 0 && nodes[i] >= nodes[i + 1]; i--);
5     if (i >= 0){
6         for (j = n - 1; nodes[j] <= nodes[i]; j--);
7         std::swap(nodes[i], nodes[j]);
8         std::reverse(nodes.begin() + i + 1, nodes.end());
9     }
10    else
11        res = false;
12    return res;
13 }
14
15 std::vector<int> brute_force(const std::vector<std::vector<double>>& dist){
16     int n = dist.size(), cur;
17     std::vector<int> min_path;
18     double min_dist = MAX_CONSTANT, cur_dist;
19     bool exists, next = true;
20     std::vector<int> allowed;
21     for (int i = 1; i < n; i++)
22         allowed.push_back(i);
23     while (next){
24         exists = true;
25         cur_dist = 0;
26         cur = 0;
27         for (int j: allowed){
28             if (dist[cur][j] < 0){
29                 exists = false;
30                 break;
31             }
32             else {
33                 cur_dist += dist[cur][j];
34                 cur = j;
35             }
36         }
37         if (exists && dist[cur][0] >= 0){
38             cur_dist += dist[cur][0];
39             if (cur_dist < min_dist){
40                 min_dist = cur_dist;
41                 min_path = {0};
42                 min_path.insert(min_path.end(), allowed.begin(), allowed.end());
43                 min_path.push_back(0);
44             }
45         }
46         next = my_next_permutation(allowed);
47     }
48     return min_path;
49 }
```

Листинг 1 – Реализация с полным перебором.

```
1 void min_dist_update(const std::vector<std::vector<int>>& ant_path, const
    std::vector<double>& ant_dist,
2     std::vector<int>& min_path, double& min_dist){
3     int m = ant_path.size();
4     for (int i = 0; i < m; i++){
5         if (ant_dist[i] >= 0){
6             if (min_path.empty() || ant_dist[i] < min_dist){
7                 min_dist = ant_dist[i];
8                 min_path = ant_path[i];
9             }
10        }
11    }
12 }
13
14 double calc_path(const int index, const std::vector<std::vector<double>>& dist, const
    std::vector<std::vector<double>>& phero,
15     std::vector<int>& ant_path, const double a, const double b){
16     ant_path.clear();
17     int n = dist.size(), cur = index, nxt;
18     std::vector<bool> visited(n, false);
19     visited[cur] = true;
20     double ant_dist = 0;
21     for (int i = 1; i < n; i++){
22         std::vector<int> allowed;
23         for (int j = 0; j < n; j++){
24             if (!visited[j] && dist[cur][j] >= 0)
25                 allowed.push_back(j);
26         }
27         if (allowed.empty()){
28             ant_dist = -1;
29             ant_path.clear();
30             break;
31         }
32         else{
33             double w_tmp, et;
34             std::vector<double> w;
35             for (int j: allowed){
36                 et = ((dist[cur][j] >= 0) ? (1 / dist[cur][j]) : 0);
37                 w_tmp = std::pow(phero[cur][j], a) * std::pow(et, b);
38                 if (w_tmp < MIN_CONSTANT)
39                     w_tmp = MIN_CONSTANT;
40                 w.push_back(w_tmp);
41                 w_sum += w_tmp;
42             }
43             std::uniform_real_distribution<double> dist01(0.0, w_sum);
44             double r = dist01(rng), acc = 0.0;
45             nxt = allowed.back();
46             for (int k = 0; k < allowed.size(); k++) {
47                 acc += w[k];
48                 if (r <= acc) {
49                     nxt = allowed[k];
50                     break;
51                 }
52             }
53             ant_dist += dist[cur][nxt];
54             ant_path.push_back(nxt);
```

```

55         visited[nxt] = true;
56         cur = nxt;
57     }
58 }
59 if (dist[cur][index] >= 0){
60     ant_dist += dist[cur][index];
61     ant_path.insert(ant_path.begin(), index);
62     ant_path.push_back(index);
63 }
64 else {
65     ant_dist = -1;
66     ant_path.clear();
67 }
68 return ant_dist;
69 }
70
71 void phero_update(std::vector<std::vector<double>>& phero, const
std::vector<std::vector<int>>& ant_path,
72 const std::vector<double>& ant_dist, const double Q, const double rho){
73     int n = phero.size(), m = ant_path.size();
74     for (int i = 0; i < n; i++){
75         for (int j = 0; j < n; j++){
76             phero[i][j] *= (1 - rho);
77         }
78         for (int k = 0; k < m; k++){
79             if (ant_dist[k] > 0){
80                 for (int t = 0; t < ant_path[k].size() - 1; t++){
81                     phero[ant_path[k][t]][ant_path[k][t + 1]] += Q / ant_dist[k];
82                 }
83             }
84         }
85     }
86 }
87
88 std::vector<int> ant_algorithm(std::vector<std::vector<double>>& dist){
89     int n = dist.size(); // city_num
90     int m = n; // ant_num
91     std::vector<std::vector<double>> phero(n, std::vector<double>(n, MIN_CONSTANT)); //
pheromones
92     double Q = static_cast<double>(n * 100); // max pheromone per cycle
93     std::vector<std::vector<int>> ant_path(m);
94     std::vector<double> ant_dist(m);
95     std::vector<int> min_path;
96     double min_dist = MAX_CONSTANT;
97     std::uniform_int_distribution<int> city_dist(0, n - 1);
98     int i;
99     for (int t = 0; t < TMAX; t++){
100         for (int k = 0; k < m; k++){
101             i = city_dist(rng);
102             ant_dist[k] = calc_path(i, dist, phero, ant_path[k], ALPHA, BETA);
103         }
104         phero_update(phero, ant_path, ant_dist, Q, RHO);
105         min_dist_update(ant_path, ant_dist, min_path, min_dist);
106     }
107     return min_path;
108 }

```

Листинг 2 – Реализация с муравьиным алгоритмом.

3.2 Тестирование

Результаты тестирования представлены в таблице 3.1.

Таблица 3.1 — Результаты тестирования.

Алгоритм	Вход	Выход
Полный перебор	$\begin{pmatrix} 0 & 2 & 9 & 10 & 7 \\ 1 & 0 & 6 & 4 & 3 \\ 9 & 7 & 0 & 8 & 5 \\ 6 & 3 & 2 & 0 & 4 \\ 3 & 8 & 5 & 6 & 0 \end{pmatrix}$	0 1 3 2 4 0
Муравьиный	$\begin{pmatrix} 0 & 2 & 9 & 10 & 7 \\ 1 & 0 & 6 & 4 & 3 \\ 9 & 7 & 0 & 8 & 5 \\ 6 & 3 & 2 & 0 & 4 \\ 3 & 8 & 5 & 6 & 0 \end{pmatrix}$	4 0 1 3 2 4

Начало и конец найденного муравьиным алгоритмом пути совпадают, следовательно результат верный.

Вывод

В данном разделе мной были реализованы описанные ранее алгоритмы и проведено их успешное тестирование.

4 Исследовательская часть

4.1 Параметризация муравьиного алгоритма

Результаты параметризации муравьиного алгоритма по параметрам ρ , α и t_{max} приведены в таблице 4.1. По полученным данным t_{max} должен лежать в диапазоне $[200, 400]$, а ρ – $[0.1, 0.25]$. Значение α не оказывает особое влияние на итоговый результат. Наиболее оптимальным является следующий набор:

- $\rho = 0.25$;
- $\alpha = 0.9$;
- $t_{max} = 200$.

4.2 Замеры реального времени

Замеры проводились на ноутбуке Redmibook Pro 14 2022:

- процессор Intel i7-12650H (архитектура x86-64, 10 ядер по 2.3 ГГц);
- оперативная память 16 Гб (DDR4, 5200 МГц);
- операционная система Windows 11 Home 25H2.

Для замеров использовалась функция `pow()` из библиотеки `chrono` [1, с. 705], результаты представлены на рис. 4.1.

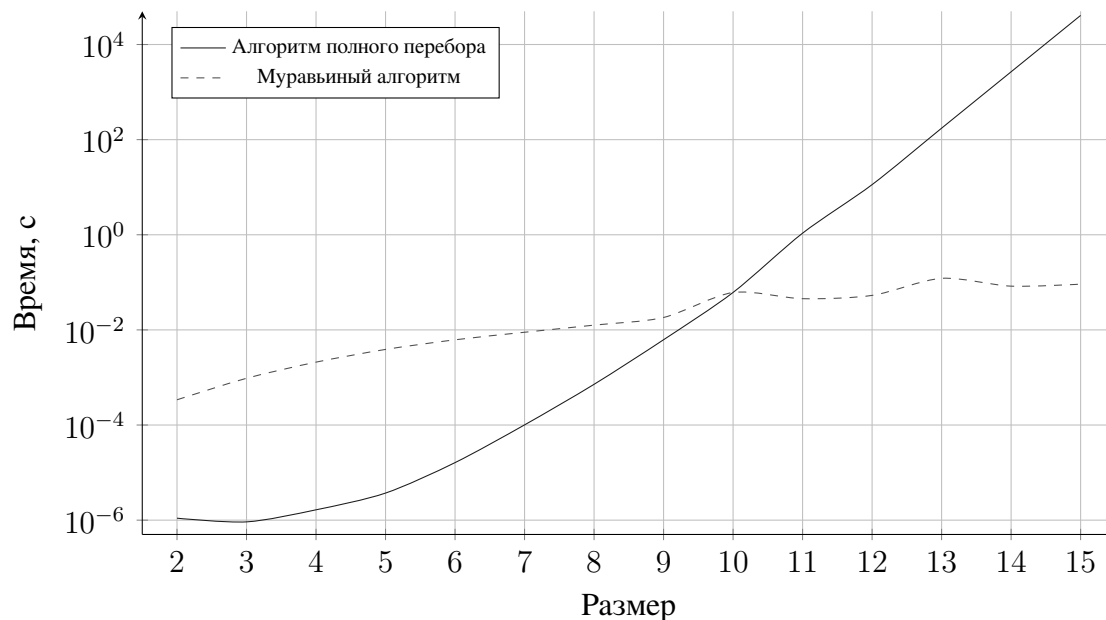


Рисунок 4.1 — Графики зависимости времени выполнения от размера графа.

Вывод

В данном разделе была проведена параметризация муравьиного алгоритма и получена наиболее оптимальная комбинация его параметров. Также были проведены замеры времени

работы для каждой реализации из предыдущего раздела. Полученные на основе этих замеров графики подтверждают рассчитанные ранее трудоёмкости рассматриваемых в работе алгоритмов.

ЗАКЛЮЧЕНИЕ

В ходе работы мной были описаны и реализованы алгоритмы полного перебора и муравьиный, вычислена их трудоёмкость. Также мной была выполнена параметризация муравьиного алгоритма. Затем мной проведен сравнительный анализ этих алгоритмов. По результатам анализа муравьиный алгоритм оказался наиболее оптимальным для обработки графов с количеством вершин более 10.

Таким образом, все задачи были выполнены, а поставленная цель достигнута.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. C++17 standard (ISO/IEC DIS 14882:2017). [Электронный ресурс] Режим доступа: <https://web.archive.org/web/20170325025026/http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/n4660.pdf> (дата обращения: 20.11.2025).
2. Галяутдинов Р. Задача коммивояжера — метод ветвей и границ. // Галяутдинов Р.Р. Сайт преподавателя экономики. – 2023. URL: <https://galyautdinov.ru/post/zadacha-kommivoyazhera> (дата обращения: 10.01.2026).
3. Полянин М. Решаем задачу коммивояжёра простым перебором. // КОД – 2024. URL: <https://thecode.media/path-js/> (дата обращения: 10.01.2026)
4. Штовба С. Д. Муравьиные алгоритмы. // Exponenta Pro. Математика в приложениях. – 2003. – №4. – С. 70–75.

ПРИЛОЖЕНИЕ

Таблица 4.1 — Результаты параметризации муравьиного алгоритма.

t_{max}	ρ	α	Граф №1			Граф №2			Граф №3			Итог		
			Макс.откл.	Ср.откл.	Мед.откл.	Макс.откл.	Ср.откл.	Мед.откл.	Макс.откл.	Ср.откл.	Мед.откл.	1	2	3
200	0.1	0.1	1765.1	1576.65	1585.45	1.43904e+10	1.22318e+10	1.25916e+10	29670.8	28474.5	28745.7	1765.1	1.43904e+10	29670.8
200	0.1	0.25	1865.48	1622.75	1655.62	1.43904e+10	1.18721e+10	1.25916e+10	29806.3	28812	29170.6	1865.48	1.43904e+10	29806.3
200	0.1	0.5	1852.78	1659.44	1647.12	1.43904e+10	1.25916e+10	1.25916e+10	30064.2	28730.1	29026.3	1865.48	1.43904e+10	30064.2
200	0.1	0.75	1815.69	1592.24	1643.34	1.43904e+10	1.22318e+10	1.25916e+10	29873.3	28879.5	28936.9	1865.48	1.43904e+10	30064.2
200	0.1	0.9	1924.84	1596.72	1513.32	1.43904e+10	1.33111e+10	1.43904e+10	30796.6	29123.6	29373.1	1924.84	1.43904e+10	30796.6
200	0.25	0.1	2177.63	1975.36	1994.4	1.43904e+10	1.22318e+10	1.25916e+10	30369.4	29183.6	29468.1	2177.63	1.43904e+10	30796.6
200	0.25	0.25	2233.18	1989.82	2000.22	1.43904e+10	1.25916e+10	1.25916e+10	30242.5	29229.3	29357.2	2233.18	1.43904e+10	30796.6
200	0.25	0.5	2085.32	1948.9	1989.73	1.61892e+10	1.29513e+10	1.25916e+10	30252.7	28901.6	28969.1	2233.18	1.61892e+10	30796.6
200	0.25	0.75	2200.27	2017.49	2083.02	1.61892e+10	1.31312e+10	1.25916e+10	30762.5	29428.3	29564.9	2233.18	1.61892e+10	30796.6
200	0.25	0.9	2204.33	2039.54	2038.7	1.43904e+10	1.27715e+10	1.25916e+10	29386	28367.4	28809.9	2233.18	1.61892e+10	30796.6
200	0.5	0.1	2435.33	2212.43	2205.65	1.43904e+10	1.29513e+10	1.25916e+10	29784.1	29067.2	29184.3	2435.33	1.61892e+10	30796.6
200	0.5	0.25	2520.47	2243.75	2340.56	1.61892e+10	1.25916e+10	1.25916e+10	29970.5	29154.7	29275.3	2520.47	1.61892e+10	30796.6
200	0.5	0.5	2525.8	2316.44	2311.44	1.43904e+10	1.20519e+10	1.25916e+10	30570.1	28981.5	29249.7	2525.8	1.61892e+10	30796.6
200	0.5	0.75	2528.59	2340.6	2380.07	1.61892e+10	1.40306e+10	1.43904e+10	30481.4	28872.3	29208.1	2528.59	1.61892e+10	30796.6
200	0.5	0.9	2546.13	2343.74	2376.59	1.61892e+10	1.33111e+10	1.43904e+10	30354.4	29202.3	29420.1	2546.13	1.61892e+10	30796.6
200	0.75	0.1	2638.52	2456.25	2490.65	1.43904e+10	1.25916e+10	1.25916e+10	30478.4	29372.8	29392	2638.52	1.61892e+10	30796.6
200	0.75	0.25	2585.33	2347.08	2361.47	1.43904e+10	1.27715e+10	1.25916e+10	30690.1	29524.9	29849.1	2638.52	1.61892e+10	30796.6
200	0.75	0.5	2666.89	2472.05	2491.31	1.43904e+10	1.33111e+10	1.25916e+10	29566.6	28078.7	28209.4	2666.89	1.61892e+10	30796.6
200	0.75	0.75	2623.96	2460.7	2530.01	1.43904e+10	1.27715e+10	1.25916e+10	30134	28645.2	29369.7	2666.89	1.61892e+10	30796.6
200	0.75	0.9	2586.84	2398.28	2470.31	1.43904e+10	1.36709e+10	1.43904e+10	29650.3	29034	29402	2666.89	1.61892e+10	30796.6
200	0.9	0.1	2560.42	2386.5	2412.33	1.43904e+10	1.29513e+10	1.25916e+10	30771	29341.9	29240.2	2666.89	1.61892e+10	30796.6
200	0.9	0.25	2645.02	2456.26	2459.35	1.43904e+10	1.24117e+10	1.25916e+10	30285.6	28847.6	28885.5	2666.89	1.61892e+10	30796.6
200	0.9	0.5	2639.81	2431.21	2448.69	1.43904e+10	1.11525e+10	1.07928e+10	29717.2	28849.3	29153.2	2666.89	1.61892e+10	30796.6
200	0.9	0.75	2550.57	2384.63	2514.34	1.43904e+10	1.22318e+10	1.25916e+10	30474.2	28181.2	28137.8	2666.89	1.61892e+10	30796.6
200	0.9	0.9	2605.41	2418.62	2456.13	1.61892e+10	1.33111e+10	1.25916e+10	29839.6	28937.5	29148.2	2666.89	1.61892e+10	30796.6
400	0.1	0.1	1541.73	1442.3	1444.53	1.43904e+10	1.33111e+10	1.43904e+10	29399.5	28047.9	28189.3	2666.89	1.61892e+10	30796.6
400	0.1	0.25	1574.8	1458.05	1446.18	1.43904e+10	1.20519e+10	1.25916e+10	29928.2	28543.8	28350.5	2666.89	1.61892e+10	30796.6
400	0.1	0.5	1753.52	1507.58	1446.18	1.25916e+10	1.16922e+10	1.25916e+10	29296.9	28083.9	28375.4	2666.89	1.61892e+10	30796.6
400	0.1	0.75	1570.46	1456.23	1446.18	1.25916e+10	1.18721e+10	1.25916e+10	28767.8	27903.3	28073	2666.89	1.61892e+10	30796.6
400	0.1	0.9	1530.46	1453.14	1446.18	1.25916e+10	1.18721e+10	1.25916e+10	29808.4	27920.9	28031	2666.89	1.61892e+10	30796.6
400	0.25	0.1	2036.07	1915.59	1978	1.43904e+10	1.20519e+10	1.25916e+10	29871.3	28267.9	28340.9	2666.89	1.61892e+10	30796.6
400	0.25	0.25	2005.38	1943.23	1953.86	1.43904e+10	1.22318e+10	1.25916e+10	29886.8	29025.4	29222	2666.89	1.61892e+10	30796.6
400	0.25	0.5	2080.46	1910.37	1905.64	1.25916e+10	1.18721e+10	1.25916e+10	28719.9	28071.3	28431.5	2666.89	1.61892e+10	30796.6
400	0.25	0.75	2123.82	1957.78	1960.24	1.25916e+10	1.24117e+10	1.25916e+10	30585.2	28420.4	28470.2	2666.89	1.61892e+10	30796.6
400	0.25	0.9	2101.93	1950.79	1997.33	1.43904e+10	1.20519e+10	1.25916e+10	29899.9	27595.8	28142.6	2666.89	1.61892e+10	30796.6
400	0.5	0.1	2419.86	2220.14	2199.13	1.43904e+10	1.24117e+10	1.25916e+10	29323.2	28531.1	28707.3	2666.89	1.61892e+10	30796.6
400	0.5	0.25	2517.13	2219.97	2227.79	1.25916e+10	1.18721e+10	1.25916e+10	30202.5	28229.9	28574.5	2666.89	1.61892e+10	30796.6
400	0.5	0.5	2436.56	2251.46	2314.52	1.43904e+10	1.22318e+10	1.25916e+10	29171.8	28197.6	28655.2	2666.89	1.61892e+10	30796.6
400	0.5	0.75	2448.29	2292.07	2300.81	1.43904e+10	1.22318e+10	1.25916e+10	29804.1	28391.8	28969.1	2666.89	1.61892e+10	30796.6
400	0.5	0.9	2387.86	2245.83	2303.88	1.43904e+10	1.15123e+10	1.07928e+10	28785.4	28179.1	28463	2666.89	1.61892e+10	30796.6
400	0.75	0.1	2478.86	2312.83	2382.08	1.43904e+10	1.24117e+10	1.25916e+10	30274.6	28457.5	28358.2	2666.89	1.61892e+10	30796.6
400	0.75	0.25	2528.08	2374.26	2404.77	1.25916e+10	1.13324e+10	1.07928e+10	29438.4	28774.3	28788	2666.89	1.61892e+10	30796.6
400	0.75	0.5	2570.32	2294.22	2309.23	1.43904e+10	1.22318e+10	1.25916e+10	29267.7	28299.7	28961.7	2666.89	1.61892e+10	30796.6
400	0.75	0.75	2489.08	2310.83	2334.1	1.43904e+10	1.25916e+10	1.25916e+10	29732.2	28549.7	28611.3	2666.89	1.61892e+10	30796.6
400	0.75	0.9	2479.89	2258.46	2286.44	1.43904e+10	1.13324e+10	1.07928e+10	29888.8	28274.5	28615	2666.89	1.61892e+10	30796.6
400	0.9	0.1	2520.57	2382.35	2423.96	1.43904e+10	1.22318e+10	1.25916e+10	29628.6	28502.6	28888.8	2666.89	1.61892e+10	30796.6
400	0.9	0.25	2572.49	2342.71	2354.82	1.43904e+10	1.18721e+10	1.25916e+10	29540.2	28921.1	29050.4	2666.89	1.61892e+10	30796.6
400	0.9	0.5	2587.47	2348.22	2378.49	1.25916e+10	1.18721e+10	1.25916e+10	29665.8	28484.4	28483.8	2666.89	1.61892e+10	30796.6
400	0.9	0.75	2575.51	2357.71	2351.99	1.43904e+10	1.25916e+10	1.25916e+10	29433.5	28707.8	28702.3	2666.89	1.61892e+10	30796.6
400	0.9	0.9	2537.9	2402.53	2421.25	1.43904e+10	1.15123e+10	1.07928e+10	29816.6	28414	28527	2666.89	1.61892e+10	30796.6
600	0.1	0.1	1495.15	1422.71	1405.65	1.25916e+10	1.09727e+10	1.07928e+10	29445.9	28201.6	28498.6	2666.89	1.61892e+10	30796.6
600	0.1	0.25	1734.5	1468.88	1405.65	1.43904e+10	1.25916e+10	1.25916e+10	29128.7	28334.1	28780.8	2666.89	1.61892e+10	30796.6
600	0.1	0.5	1493.82	1433.9	1446.18	1.43904e+10	1.18721e+10	1.25916e+10	28813.7	27992.7	28548.1	2666.89	1.61892e+10	30796.6
600	0.1	0.75	1550.71	1436.2	1444.53	1.25916e+10	1.13324e+10	1.25916e+10	29519.2	27661.2	28065.8	2666.89	1.61892e+10	30796.6
600	0.1	0.9	1446.18	1425.22	1439.25	1.25916e+10	1.15123e+10	1.25916e+10	29590.5	27885.1	28143.5	2666.89	1.61892e+10	30796.6
600	0.25	0.1	2030.99	1830.53	1861.47	1.25916e+10	1.09727e+10	1.07928e+10	29296.8	27988.2	28253.3	2666.89	1.61892e+10	30796.6
600	0.25	0.25	2066.75	1918.79	1967.27	1.25916e+10	1.24117e+10	1.25916e+10	29134.3	28207.8	28364	2666.89	1.61892e+10	30796.6
600	0.25	0.5	2043.39	1860.05	1963.11	1.25916e+10	1.16922e+10	1.25916e+10	29036.6	28016.9	28066.2	2666.89	1.61892e+10	30796.6
600	0.25	0.75	2134.84	1928.01	1949.02	1.43904e+10	1.22318e+10	1.25916e+10	28447.1	27697.9	27990.3	2666.89	1.61892e+10	30796.6
600	0.25	0.9	2018.36	1903.13	1881.21	1.25916e+10	1.18721e+10	1.25916e+10	28901.8	27536.2	27846.5	2666.89	1.61892e+10	30796.6
600	0.5	0.1	2442.8	2159.04	2169.46	1.25916e+10	1.11525e+10	1.07928e+10	28910.9	27598.5	27511	2666.89	1.61892e+10	30796.6
600	0.5	0.25	2343.71	2166.77	2195.51	1.25916e+10	1.15123e+10	1.25916e+10	29642.2	28492.9	28723.5	2666.89	1.61892e+10	30796.6
600	0.5	0.5	2296.31	2131.08	2142.52	1.25916e+10	1.16922e+10	1.25916e+10	29229.6	28163.2	28610.4	2666.89	1.61892e+10	30796.6
600	0.5													

Таблица 4.2 — Результаты параметризации муравьиного алгоритма – продолжение.

t_{max}	ρ	α	Граф №1			Граф №2			Граф №3			Итог		
			Макс.откл.	Ср.откл.	Мед.откл.	Макс.откл.	Ср.откл.	Мед.откл.	Макс.откл.	Ср.откл.	Мед.откл.	1	2	3
800	0.5	0.1	2261.52	2148.63	2155.28	1.25916e+10	1.15123e+10	1.07928e+10	28953.1	28011	28414.4	2699.34	1.61892e+10	30796.6
800	0.5	0.25	2247.62	2081.21	2184.01	1.25916e+10	1.09727e+10	1.07928e+10	28581.4	27439.1	27493.4	2699.34	1.61892e+10	30796.6
800	0.5	0.5	2277.53	2141.54	2155.67	1.25916e+10	1.06129e+10	1.07928e+10	28546.8	27651.7	28139	2699.34	1.61892e+10	30796.6
800	0.5	0.75	2339.27	2195.42	2233.99	1.43904e+10	1.16922e+10	1.25916e+10	28761.2	27712.5	27590.9	2699.34	1.61892e+10	30796.6
800	0.5	0.9	2357.09	2125.38	2103.31	1.25916e+10	1.13324e+10	1.07928e+10	28742.4	27458	27739.4	2699.34	1.61892e+10	30796.6
800	0.75	0.1	2426.92	2237.19	2271.69	1.25916e+10	1.16922e+10	1.25916e+10	28688.6	27557.3	28017.2	2699.34	1.61892e+10	30796.6
800	0.75	0.25	2491.32	2313.66	2373.45	1.25916e+10	1.16922e+10	1.25916e+10	28950.5	28005.5	28152.6	2699.34	1.61892e+10	30796.6
800	0.75	0.5	2512.03	2178.42	2257.55	1.25916e+10	1.06129e+10	1.07928e+10	28529.2	27985.4	27981	2699.34	1.61892e+10	30796.6
800	0.75	0.75	2484.51	2240.17	2264.56	1.25916e+10	1.13324e+10	1.25916e+10	28750.2	27173.5	27306.6	2699.34	1.61892e+10	30796.6
800	0.75	0.9	2388.99	2264.56	2292.06	1.25916e+10	1.06129e+10	1.07928e+10	29163.5	27877.7	27889.4	2699.34	1.61892e+10	30796.6
800	0.9	0.1	2597.5	2298.05	2276.2	1.25916e+10	1.18721e+10	1.25916e+10	28638.4	27673.9	27911	2699.34	1.61892e+10	30796.6
800	0.9	0.25	2432.62	2242.66	2264.3	1.25916e+10	1.09727e+10	1.07928e+10	28999.8	27477.5	27630.7	2699.34	1.61892e+10	30796.6
800	0.9	0.5	2355.22	2272.78	2291.09	1.25916e+10	1.13324e+10	1.07928e+10	30167.2	28119.1	27934	2699.34	1.61892e+10	30796.6
800	0.9	0.75	2412.16	2245.44	2277.51	1.25916e+10	1.13324e+10	1.25916e+10	29559.9	28332.2	28399.1	2699.34	1.61892e+10	30796.6
800	0.9	0.9	2464.17	2301.08	2345.18	1.25916e+10	1.06129e+10	1.07928e+10	29263.1	28261.1	28337.1	2699.34	1.61892e+10	30796.6
1000	0.1	0.1	1405.65	1405.17	1405.65	1.25916e+10	1.07928e+10	1.07928e+10	29027.6	27780	27808.1	2699.34	1.61892e+10	30796.6
1000	0.1	0.25	1446.18	1406.69	1405.65	1.25916e+10	1.09727e+10	1.07928e+10	28437.3	27601.8	27808.7	2699.34	1.61892e+10	30796.6
1000	0.1	0.5	1446.18	1409.7	1405.65	1.25916e+10	1.15123e+10	1.07928e+10	29461.9	27291.9	27091.3	2699.34	1.61892e+10	30796.6
1000	0.1	0.75	1446.18	1413.76	1405.65	1.25916e+10	1.11525e+10	1.07928e+10	28752.2	28004.2	28194.7	2699.34	1.61892e+10	30796.6
1000	0.1	0.9	1446.18	1419.6	1405.65	1.25916e+10	1.07928e+10	1.07928e+10	28681.9	27203.5	27222.5	2699.34	1.61892e+10	30796.6
1000	0.25	0.1	2117.42	1829.14	1795.33	1.25916e+10	1.07928e+10	1.07928e+10	29985.4	28116.4	28336.1	2699.34	1.61892e+10	30796.6
1000	0.25	0.25	1968.55	1861.96	1890.78	1.25916e+10	1.13324e+10	1.07928e+10	27851.4	27209.8	27307.2	2699.34	1.61892e+10	30796.6
1000	0.25	0.5	1969.13	1832.86	1813.54	1.25916e+10	1.15123e+10	1.25916e+10	28958	28147	28388.1	2699.34	1.61892e+10	30796.6
1000	0.25	0.75	1921.77	1807.11	1848.18	1.25916e+10	1.11525e+10	1.07928e+10	28773.1	27337	27267.7	2699.34	1.61892e+10	30796.6
1000	0.25	0.9	2044.73	1847.96	1896.37	1.25916e+10	1.15123e+10	1.07928e+10	28996.3	27594.5	27883.4	2699.34	1.61892e+10	30796.6
1000	0.5	0.1	2289.29	2040.84	2082.25	1.43904e+10	1.15123e+10	1.07928e+10	28156.4	27321.1	27491.9	2699.34	1.61892e+10	30796.6
1000	0.5	0.25	2342.03	2116.06	2087.5	1.25916e+10	1.11525e+10	1.07928e+10	28583	27919.8	28360.8	2699.34	1.61892e+10	30796.6
1000	0.5	0.5	2273.24	2101.52	2122.81	1.25916e+10	1.16922e+10	1.25916e+10	28367.3	27718.2	27890.5	2699.34	1.61892e+10	30796.6
1000	0.5	0.75	2243.91	2083.73	2148.93	1.25916e+10	1.07928e+10	1.07928e+10	28814.6	27733.8	28083.3	2699.34	1.61892e+10	30796.6
1000	0.5	0.9	2285.48	2113.98	2179.35	1.25916e+10	1.06129e+10	1.07928e+10	28698.3	27578.4	28014.1	2699.34	1.61892e+10	30796.6
1000	0.75	0.1	2428.39	2215.06	2251.25	1.07928e+10	1.00733e+10	1.07928e+10	28652.7	27812.7	27750	2699.34	1.61892e+10	30796.6
1000	0.75	0.25	2362.9	2225.83	2238.12	1.25916e+10	1.13324e+10	1.07928e+10	28515.7	27373.6	27762.5	2699.34	1.61892e+10	30796.6
1000	0.75	0.5	2402.3	2166.65	2167.62	1.25916e+10	1.15123e+10	1.25916e+10	29248.9	27981.6	27936.6	2699.34	1.61892e+10	30796.6
1000	0.75	0.75	2372.43	2215.95	2239.51	1.25916e+10	1.13324e+10	1.07928e+10	28721.8	26933.6	27671.9	2699.34	1.61892e+10	30796.6
1000	0.75	0.9	2374.98	2191.61	2240.96	1.25916e+10	1.09727e+10	1.07928e+10	28979	27772	27868.7	2699.34	1.61892e+10	30796.6
1000	0.9	0.1	2472.21	2260.14	2270.81	1.25916e+10	1.09727e+10	1.07928e+10	28908.8	27484.8	27668.5	2699.34	1.61892e+10	30796.6
1000	0.9	0.25	2462.73	2203.48	2186.6	1.25916e+10	1.11525e+10	1.07928e+10	28709.3	27552.4	27927.4	2699.34	1.61892e+10	30796.6
1000	0.9	0.5	2372.33	2166.74	2176.08	1.25916e+10	1.11525e+10	1.07928e+10	29217.3	28259.8	28176.5	2699.34	1.61892e+10	30796.6
1000	0.9	0.75	2482.46	2295.77	2336.74	1.25916e+10	1.09727e+10	1.07928e+10	28803.9	27726.7	28095.3	2699.34	1.61892e+10	30796.6
1000	0.9	0.9	2449.21	2298.55	2329.69	1.25916e+10	1.11525e+10	1.25916e+10	29055.8	27900.8	27873.8	2699.34	1.61892e+10	30796.6